

EngVid Learning Tracker

Documento Maestro de Proyecto v1.0

INFORMACIÓN DEL PROYECTO

Nombre del Proyecto: EngVid Learning Tracker
Versión: MVP 1.0
Fecha de Inicio: Diciembre 15, 2025
Fecha Objetivo MVP: Enero 15, 2026 (4 semanas)
Estado:  En Planificación

EQUIPO DE DESARROLLO

Rol	Nombre	Responsabilidades	Horas/Semana
Full Stack Developer	[Tu nombre]	Backend, Base de Datos, DevOps	5 horas
Full Stack Developer	[Nombre hermano]	Frontend, Base de Datos, Testing	5 horas

Horario de Trabajo: Lunes a Viernes
Reunión Semanal: Jueves por la tarde
Metodología: Sprints semanales (4 sprints totales)

OBJETIVOS DEL PROYECTO

Objetivo Principal

Crear una aplicación web mobile-first que permita a usuarios de EngVid hacer seguimiento de su progreso de aprendizaje, visualizar videos, registrar calificaciones y mantener rachas de estudio.

Objetivos Secundarios

- Desarrollar un proyecto profesional para portfolio
- Aprender desarrollo Full Stack con tecnologías modernas
- Potencialmente ofrecer la herramienta a EngVid.com

4. Aplicar mejores prácticas de la industria

Métricas de Éxito

- ☐ Sistema funcional con usuarios registrados
 - ☐ Visualización de videos con filtros
 - ☐ Sistema de rachas operativo
 - ☐ Dashboards con métricas en tiempo real
 - ☐ Deploy exitoso en producción
 - ☐ Testing automatizado básico implementado
-

ARQUITECTURA TÉCNICA

Stack Tecnológico Recomendado

Backend

- **Framework:** Flask (Python 3.10+)
- **ORM:** SQLAlchemy
- **Autenticación:** Flask-JWT-Extended + OAuth2 (Google)
- **Validación:** Flask-Mail para verificación de email
- **Base de Datos:** MySQL 8.0
- **Testing:** pytest

Frontend

- **Base:** HTML5 + CSS3 + JavaScript Vanilla
- **Styling:** Tailwind CSS (CDN para MVP)
- **Componentes:** Web Components nativos
- **HTTP Client:** Fetch API
- **Testing:** Jest (básico)

Infraestructura

- **Hosting Backend:** Railway.app (gratis, fácil setup, MySQL incluido)
- **Hosting Frontend:** Netlify (gratis, deploy automático desde GitHub)

- **Control de Versiones:** GitHub (repositorio privado inicialmente)
- **CI/CD:** GitHub Actions (básico)

Gestión de Proyecto

- **Tablero:** Trello
 - **Comunicación:** WhatsApp/Slack
 - **Documentación:** GitHub Wiki + README.md
-

PRODUCT BACKLOG PRIORIZADO

PRIORIDAD CRÍTICA (MVP Core)

EPIC 1: Sistema de Usuarios

- **US-001:** Como usuario nuevo, quiero registrarme con email/password para acceder a la plataforma
 - Validación de email obligatoria
 - Hash de contraseñas con bcrypt
 - Criterio de aceptación: Usuario recibe email y puede activar cuenta
- **US-002:** Como usuario, quiero hacer login con Google para acceder rápidamente
 - OAuth2 con Google
 - Crear usuario automáticamente si no existe
- **US-003:** Como usuario, quiero recuperar mi contraseña si la olvido
 - Email con token temporal
 - Expiración del token en 1 hora

EPIC 2: Gestión de Videos

- **US-004:** Como usuario, quiero ver un video aleatorio para estudiar algo nuevo
 - Botón "Video Aleatorio"
 - Excluir videos ya vistos (opcional en filtros)
 - Mostrar: título, nivel, categoría, enlace a EngVid
- **US-005:** Como usuario, quiero filtrar videos por nivel para estudiar a mi ritmo

- Dropdown: Beginner / Intermediate / Advanced
- Mostrar conteo de videos disponibles
- **US-006:** Como usuario, quiero filtrar videos por categoría para enfocarme en temas específicos
 - Dropdown con 14 categorías
 - Permitir múltiples filtros simultáneos
- **US-007:** Como usuario, quiero marcar un video como "visto" para llevar control
 - Botón "Marcar como visto"
 - Registrar fecha/hora automáticamente
 - Confirmación visual
- **US-008:** Como usuario, quiero registrar mi calificación del quiz para ver mi progreso
 - Input numérico (0-10)
 - Guardar solo si es mayor al score previo
 - Mostrar histórico de intentos

EPIC 3: Sistema de Rachas

- **US-009:** Como usuario, quiero ver mi racha actual de estudio para mantenerme motivado
 - Contador visible en dashboard
 - Ícono de fuego/llama
 - Actualización en tiempo real
- **US-010:** Como usuario, quiero ver mi racha más larga histórica para superar mi récord
 - Badge especial si se iguala/supera
 - Fecha de inicio de la racha más larga

EPIC 4: Dashboard Personal

- **US-011:** Como usuario, quiero ver mis estadísticas personales para conocer mi progreso
 - Total de videos vistos
 - Promedio de calificaciones (últimos 10 videos)
 - Videos por nivel completados (gráfica de barras)
 - Racha actual y récord
 - Categorías más estudiadas (top 5)

EPIC 5: Dashboard Administrativo

- **US-012:** Como administrador, quiero ver métricas de la plataforma para evaluar su uso
 - Total de usuarios registrados
 - Usuarios activos (últimos 7 días)
 - Videos más vistos (top 10)
 - Promedio general de calificaciones
 - Gráfica de nuevos registros por semana
-

PRIORIDAD MEDIA (Post-MVP)

- **US-013:** Sistema de favoritos/watchlist
 - **US-014:** Búsqueda por palabra clave en títulos
 - **US-015:** Modo oscuro/claro
 - **US-016:** Notificaciones por email (racha en peligro)
 - **US-017:** Exportar progreso a PDF/CSV
-

PRIORIDAD BAJA (Futuro)

- **US-018:** Sistema de logros/badges
 - **US-019:** Comparación con otros usuarios
 - **US-020:** Integración con API de EngVid (si existe)
-

PLANIFICACIÓN DE SPRINTS

Sprint 1: Setup y Autenticación (Semana 1)

Objetivo: Configurar el proyecto y tener sistema de usuarios funcional

Tareas:

- ☐ T1.1: Crear repositorio en GitHub
- ☐ T1.2: Configurar estructura de carpetas

- ☐ T1.3: Setup de entorno Python (Flask + dependencias)
- ☐ T1.4: Setup de MySQL local
- ☐ T1.5: Crear schema de base de datos (users, videos, progress, streaks)
- ☐ T1.6: Importar CSV de videos a MySQL (script Python)
- ☐ T1.7: Implementar endpoints de registro/login (JWT)
- ☐ T1.8: Implementar validación de email
- ☐ T1.9: Crear pantallas de login/registro (HTML)
- ☐ T1.10: Integrar frontend con backend (auth)
- ☐ T1.11: Testing manual de autenticación

Definition of Done:

- Código en GitHub
 - Usuario puede registrarse y recibir email
 - Usuario puede hacer login y recibir JWT
 - Frontend protege rutas privadas
 - Base de datos tiene los 2800 videos cargados
-

Sprint 2: Visualización y Registro de Videos (Semana 2)

Objetivo: Permitir ver videos y registrar progreso

Tareas:

- ☐ T2.1: Endpoint GET /videos/random
- ☐ T2.2: Endpoint GET /videos?level=X&category=Y
- ☐ T2.3: Endpoint POST /progress (marcar visto)
- ☐ T2.4: Endpoint POST /progress/quiz (registrar score)
- ☐ T2.5: Pantalla de visualización de video
- ☐ T2.6: Componente de filtros (nivel + categoría)
- ☐ T2.7: Botón "Video Aleatorio"
- ☐ T2.8: Formulario de registro de calificación
- ☐ T2.9: Integración completa frontend-backend
- ☐ T2.10: Testing de flujo completo

Definition of Done:

- Usuario puede obtener video aleatorio

- Usuario puede filtrar por nivel y categoría
 - Usuario puede marcar video como visto
 - Usuario puede registrar calificación
 - UI es responsive (mobile-first)
 - Código testeado manualmente
-

Sprint 3: Sistema de Rachas y Dashboard Personal (Semana 3)

Objetivo: Implementar rachas y dashboard con métricas personales

Tareas:

- ☐ T3.1: Lógica de cálculo de racha actual
- ☐ T3.2: Lógica de racha más larga histórica
- ☐ T3.3: Endpoint GET /streaks/:userId
- ☐ T3.4: Endpoint GET /dashboard/personal
- ☐ T3.5: Crear componente de visualización de racha
- ☐ T3.6: Crear dashboard personal con gráficas
- ☐ T3.7: Implementar Chart.js para visualizaciones
- ☐ T3.8: Integración frontend-backend
- ☐ T3.9: Testing de rachas (casos límite)

Definition of Done:

- Racha se actualiza correctamente cada día
 - Dashboard muestra todas las métricas definidas
 - Gráficas son legibles en móvil
 - Casos edge testeados (cambio de día, primer video)
-

Sprint 4: Dashboard Admin, Testing y Deploy (Semana 4)

Objetivo: Dashboard administrativo, testing automatizado y deployment

Tareas:

- ☐ T4.1: Endpoint GET /admin/dashboard
- ☐ T4.2: Proteger rutas de admin (middleware)

- ☐ T4.3: Crear dashboard admin con métricas
- ☐ T4.4: Implementar gráficas de uso
- ☐ T4.5: Setup de pytest (backend)
- ☐ T4.6: Escribir tests críticos (auth, videos, rachas)
- ☐ T4.7: Setup de Railway para backend
- ☐ T4.8: Setup de Netlify para frontend
- ☐ T4.9: Configurar variables de entorno en producción
- ☐ T4.10: Deploy y pruebas en producción
- ☐ T4.11: Documentación final (README + Wiki)

Definition of Done:

- Admin puede ver métricas de la plataforma
 - Tests automatizados pasan (coverage >60%)
 - Aplicación desplegada y accesible públicamente
 - Variables de entorno seguras
 - README con instrucciones de instalación
 - Video demo del funcionamiento
-

MODELO DE DATOS

Tabla: users

```
sql

id          INT PRIMARY KEY AUTO_INCREMENT
username    VARCHAR(50) UNIQUE NOT NULL
email       VARCHAR(100) UNIQUE NOT NULL
password_hash VARCHAR(255) NOT NULL
google_id   VARCHAR(100) UNIQUE NULL
is_admin    BOOLEAN DEFAULT FALSE
email_verified BOOLEAN DEFAULT FALSE
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
last_login  TIMESTAMP NULL
```

Tabla: videos

```
sql
```



```
id          INT PRIMARY KEY AUTO_INCREMENT
title       VARCHAR(255) NOT NULL
level       ENUM('Beginner', 'Intermediate', 'Advanced') NOT NULL
category    VARCHAR(100) NOT NULL
url         VARCHAR(500) NOT NULL
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

Tabla: user_video_progress

```
sql

id          INT PRIMARY KEY AUTO_INCREMENT
user_id     INT NOT NULL
video_id    INT NOT NULL
watched_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
quiz_score  INT NULL (0-10)
best_score  INT NULL (histórico más alto)
attempt_count INT DEFAULT 1
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
INDEX idx_user_video (user_id, video_id)
```

Tabla: user_streaks

```
sql

id          INT PRIMARY KEY AUTO_INCREMENT
user_id     INT NOT NULL
streak_date DATE NOT NULL
videos_watched INT DEFAULT 1
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
UNIQUE KEY unique_streak (user_id, streak_date)
INDEX idx_user_date (user_id, streak_date)
```

Tabla: email_verifications

```
sql
```

```
id          INT PRIMARY KEY AUTO_INCREMENT
user_id     INT NOT NULL
token       VARCHAR(255) UNIQUE NOT NULL
expires_at  TIMESTAMP NOT NULL
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

DEFINICIÓN DE ROLES Y PERMISOS

Usuario Regular

-  Ver videos
-  Registrar progreso
-  Ver dashboard personal
-  Ver dashboard admin
-  Gestionar usuarios

Administrador

-  Todo lo de usuario regular
 -  Ver dashboard admin
 -  Ver lista de usuarios
 -  Eliminar usuarios (no implementado en MVP)
-

GUÍA DE DISEÑO UI/UX

Principios de Diseño

1. **Mobile-First:** Diseñar primero para 375px (iPhone SE)
2. **Minimalista:** Espacios blancos, tipografía clara
3. **Accesible:** Contraste WCAG AA mínimo
4. **Rápido:** Carga <2 segundos

Paleta de Colores (Sugerencia)

- **Primario:**  #3B82F6 (Azul confianza)
- **Secundario:**  #10B981 (Verde progreso/racha)
- **Fondo:**  #F9FAFB (Gris muy claro)
- **Texto:**  #111827 (Casi negro)
- **Error:**  #EF4444 (Rojo)
- **Éxito:**  #10B981 (Verde)

Tipografía

- **Familia:** Inter (Google Fonts)
- **Títulos:** 24px - 32px, font-weight: 700
- **Cuerpo:** 16px, font-weight: 400
- **Labels:** 14px, font-weight: 500

Componentes Clave

- **Botón Primario:** Azul, redondeado, sombra sutil
- **Card de Video:** Borde sutil, hover effect
- **Badge de Racha:** Ícono  + número grande
- **Gráficas:** Simples, colores consistentes

MÉTRICAS DEL DASHBOARD

Dashboard Personal

1. **Resumen Superior:**
 - Total videos vistos (número grande)
 - Racha actual  (número + días)
 - Racha récord  (número + fecha)
 - Promedio últimos 10 quizzes (número con 1 decimal)
2. **Gráfica de Barras:** Videos por Nivel

- Beginner: X videos
- Intermediate: Y videos
- Advanced: Z videos

3. Top 5 Categorías Más Estudiadas

- Lista ordenada con conteo

4. Histórico Semanal (opcional para MVP)

- Videos vistos por día (últimos 7 días)

Dashboard Admin

1. Métricas Generales:

- Total usuarios registrados
- Usuarios activos (últimos 7 días)
- Total videos en plataforma (2800+)
- Promedio general de calificaciones

2. Top 10 Videos Más Vistos

- Título + conteo de visualizaciones

3. Gráfica de Línea: Nuevos Registros

- Por semana (últimas 8 semanas)

4. Distribución por Nivel de Estudio

- Gráfica circular: % de videos vistos por nivel

DEFINITION OF DONE (DoD)

Una tarea está "Done" cuando cumple:

Para Features de Backend:

- ☐ Código escrito y funcional
- ☐ Endpoint documentado (docstring + ejemplo)
- ☐ Manejo de errores implementado (try/catch)
- ☐ Validación de inputs
- ☐ Al menos 1 test automatizado

- ☐ Probado manualmente con Postman/curl
- ☐ Código revisado por el otro miembro del equipo
- ☐ Merge a rama `develop`

Para Features de Frontend:

- ☐ Código escrito y funcional
- ☐ Responsive en móvil (375px mínimo)
- ☐ Probado en Chrome y Firefox
- ☐ Manejo de estados de carga
- ☐ Manejo de errores (mensajes claros al usuario)
- ☐ Código revisado por el otro miembro del equipo
- ☐ Merge a rama `develop`

Para el MVP Completo:

- ☐ Todas las User Stories críticas completadas
 - ☐ Testing automatizado ejecutándose (CI)
 - ☐ Deploy en producción exitoso
 - ☐ Probado end-to-end en dispositivo móvil real
 - ☐ README.md actualizado
 - ☐ Video demo grabado (3-5 minutos)
-

ESTRATEGIA DE TESTING

Testing Backend (pytest)

Cobertura Mínima: 60%

Tests Críticos:

- `test_user_registration()` - Registro exitoso
- `test_login_jwt()` - Login y generación de token
- `test_random_video()` - Obtener video aleatorio
- `test_mark_video_watched()` - Registrar progreso
- `test_streak_calculation()` - Cálculo correcto de racha
- `test_quiz_score_update()` - Guardar solo mejor score

Testing Frontend (Manual para MVP)

Checklist:

- ☐ Registro de usuario funciona en móvil
 - ☐ Login persiste sesión (JWT en localStorage)
 - ☐ Video aleatorio carga correctamente
 - ☐ Filtros aplican bien
 - ☐ Racha se actualiza correctamente
 - ☐ Dashboard muestra datos reales
-

GLOSARIO TÉCNICO

Términos de Desarrollo

- **API (Application Programming Interface):** Conjunto de endpoints que expone el backend
- **Endpoint:** URL específica del backend que realiza una acción (ej: POST /login)
- **JWT (JSON Web Token):** Token de autenticación que se envía en cada petición
- **ORM (Object-Relational Mapping):** SQLAlchemy, convierte objetos Python en SQL
- **CORS (Cross-Origin Resource Sharing):** Permite que frontend y backend se comuniquen
- **Migration:** Script que modifica la estructura de la base de datos
- **Seed:** Datos iniciales que se cargan en la BD (ej: los 2800 videos)

Términos de Gestión

- **Sprint:** Ciclo de trabajo de 1 semana
- **User Story (US):** Funcionalidad desde la perspectiva del usuario
- **Epic:** Conjunto de User Stories relacionadas
- **MVP (Minimum Viable Product):** Versión mínima funcional del producto
- **DoD (Definition of Done):** Criterios para considerar una tarea terminada
- **Backlog:** Lista priorizada de tareas pendientes

Términos de Negocio

- **Racha (Streak):** Días consecutivos con al menos 1 video visto

- **Score:** Calificación del quiz (0-10)
- **Dashboard:** Pantalla con métricas y estadísticas
- **Nivel (Level):** Beginner, Intermediate, Advanced
- **Categoría:** Tipo de contenido (grammar, vocabulary, etc.)

RIESGOS Y MITIGACIONES

Riesgo	Probabilidad	Impacto	Mitigación
EngVid cambia estructura de URLs	Media	Alto	Guardar backup del CSV, diseñar scraper flexible
Uno del equipo no puede continuar	Baja	Alto	Documentar todo, código autoexplicativo
MySQL gratuito se queda sin espacio	Baja	Medio	Monitorear uso, Railway tiene 1GB gratis
Despliegue falla en producción	Media	Medio	Probar en staging primero, rollback plan
No se cumple el plazo de 1 mes	Media	Bajo	Reducir scope, priorizar features críticas

PROTOCOLO DE COMUNICACIÓN

Reunión Semanal (Jueves tarde)

Agenda:

1. Revisión de sprint anterior (15 min)
2. Demo de lo completado (15 min)
3. Planificación del próximo sprint (20 min)
4. Identificación de bloqueos (10 min)

Formato: Presencial o videollamada

Duración: 1 hora máximo

Comunicación Diaria

- **WhatsApp/Slack:** Updates rápidos y bloqueos
- **GitHub:** Code reviews y comentarios técnicos

- **Trello:** Mover tarjetas de estado
-

RECURSOS Y REFERENCIAS

Documentación Oficial

- Flask: <https://flask.palletsprojects.com/>
- Tailwind CSS: <https://tailwindcss.com/docs>
- MySQL: <https://dev.mysql.com/doc/>
- Railway: <https://docs.railway.app/>

Tutoriales Recomendados

- Flask Mega-Tutorial (español): <https://blog.miguelgrinberg.com/>
 - Tailwind Components: <https://tailwindui.com/components>
 - JWT Authentication: <https://jwt.io/introduction>
-

REGISTRO DE CAMBIOS

Versión	Fecha	Cambios	Autor
1.0	2025-12-15	Documento inicial creado	[Tu nombre]

CHECKLIST DE INICIO

Antes de comenzar Sprint 1:

- ☐ Documento leído y entendido por ambos
 - ☐ Trello configurado con las 4 épicas
 - ☐ GitHub repositorio creado (privado)
 - ☐ Entorno de desarrollo configurado (Python, MySQL, VS Code)
 - ☐ Reunión inicial de kickoff realizada
-

Última actualización: Diciembre 15, 2025

Próxima revisión: Diciembre 19, 2025 (Jueves - Primera reunión)